

中国科学技术大学

高质量实时渲染 技术挑战赛



基于 HarmonyOS 的透明泡泡实时折射 与色散渲染系统

队伍名称： 吃福是亏

队伍成员： 杨弘宇、常毅寒、杨硕、高凡、赵艺博

指导老师： 蔡有城

University of Science and Technology of China
CCF CAD/CG 2026 High-Quality
Real-Time Rendering Technology
Challenge



**Real-Time Transparent Bubble
Rendering with Refraction and
Iridescence on HarmonyOS**

Team Members: Hongyu Yang, Yihan Chang, Shuo Yang, Fan Gao, Yibo Zhao

Supervisor: Youcheng Cai

摘 要

本项目面向【CCF CAD/CG 2026】高质量实时渲染技术挑战赛赛题二“基于鸿蒙的透明物体实时色散渲染系统”，实现了一个运行于 HarmonyOS 设备上的透明泡泡实时渲染与交互系统。系统采用 HarmonyOS ArkTS + XComponent + NAPI + C++ OpenGL ES 3.0 的整体架构，以屏幕空间折射为核心，通过多阶段 FBO 合成近似肥皂泡的双界面折射；在材质上实现了 Kim2012 的实时薄膜干涉计算模式、基于预计算纹理的 Spectral LUT 模式，以及 Belcour Airy 风格的多波长薄膜反射模式；在动态与交互上实现了基于噪声和排液趋势的膜厚变化、触摸驱动的局部扰动与波纹、主泡泡的 DBSTT/vortex sheet 表面形变，以及多展示泡泡的漂移组织与参数面板控制。本文按照渲染管线、光学模型、交互实现与动态模拟的顺序介绍最终初赛提交版本的系统设计、关键实现与局限。

关键词：肥皂泡渲染，HarmonyOS，屏幕空间折射，薄膜干涉，虹彩，泡泡形变仿真，OpenGL ES

ABSTRACT

This project targets Track 2, “Real-Time Dispersion Rendering of Transparent Objects on HarmonyOS,” and implements a real-time transparent bubble rendering and interaction system on HarmonyOS devices. The final system is built with HarmonyOS ArkTS, XComponent, NAPI, and C++ OpenGL ES 3.0. Multi-stage framebuffer composition is used to approximate two-interface refraction, the renderer supports three iridescence modes named Kim2012, Spectral LUT, and Belcour Airy, and the interactive demo includes procedural film-thickness variation, touch-driven local disturbance, DBSTT/vortex-sheet-inspired deformation for the main bubble, and multiple drifting display bubbles with a real-time parameter panel. This report directly presents the final HarmonyOS submission, including its rendering pipeline, optical models, interaction design, simulation method, and current limitations.

Key Words: soap bubble rendering; HarmonyOS; screen-space refraction; thin-film interference; iridescence; Bubble Simulation; OpenGL ES

目 录

第 1 章 引言	1
1.1 赛题目标与总体结构	1
1.2 系统组成	2
1.3 运行环境	2
第 2 章 渲染管线设计	3
2.1 渲染管线	3
2.1.1 多阶段屏幕空间折射	3
2.1.2 FBO 尺寸策略与坐标修正	4
2.2 屏幕空间折射实现	4
2.2.1 折射方向	4
2.2.2 边缘增强与薄膜厚度影响	5
2.3 薄膜虹彩模型	6
2.3.1 薄膜干涉基础	6
2.3.2 Kim2012 模式：实时 RGB 波长采样	6
2.3.3 Spectral LUT 模式：预计算薄膜反射纹理	7
2.3.4 Belcour Airy 模式：多波长 Airy 反射近似	7
2.3.5 三种模式对比	8
2.4 动态膜厚与颜色合成	8
2.4.1 动态膜厚	8
2.4.2 颜色合成	9
第 3 章 物理模拟与动力学	10
3.1 基于涡流片的单泡泡物理模拟	10
3.1.1 涡流片模型的理论基础	10
3.1.2 离散化与数值实现	11
3.1.3 与本项目渲染管线的集成	14
3.2 触摸交互与局部光学响应	15
3.3 多泡泡展示与风吹漂移	16
第 4 章 交互与演示	18
4.1 交互实现	18
4.1.1 触控交互	18
4.1.2 界面参数	18

第 5 章 结果与分析	20
5.1 结果分析与局限	20
5.2 总结	20
参考文献	22

第 1 章 引 言

1.1 赛题目标与总体结构

赛题二“基于鸿蒙的透明物体实时色散渲染系统”要求参赛者在给定基线工程上完成透明物体的实时折射、色散与交互展示，并进一步体现多泡泡效果、运动模拟、完整 UI 和稳定运行能力。肥皂泡恰好同时包含透明折射、薄膜干涉、动态表面和交互反馈四类典型问题，因此非常适合作为赛题落地对象。

本技术文档将介绍本队最终初赛提交版本的核心技术路线。系统围绕一个主泡泡展开：主泡泡负责展示折射、虹彩、触摸扰动和表面形变；周围的展示泡泡负责增强多目标层次与现场演示观感。实现上，系统被组织为渲染管线、薄膜材质、几何数据、动态模拟和资源加载几个相对独立的模块，并通过 ArkTS 页面层与 Native 渲染层共同完成最终应用。

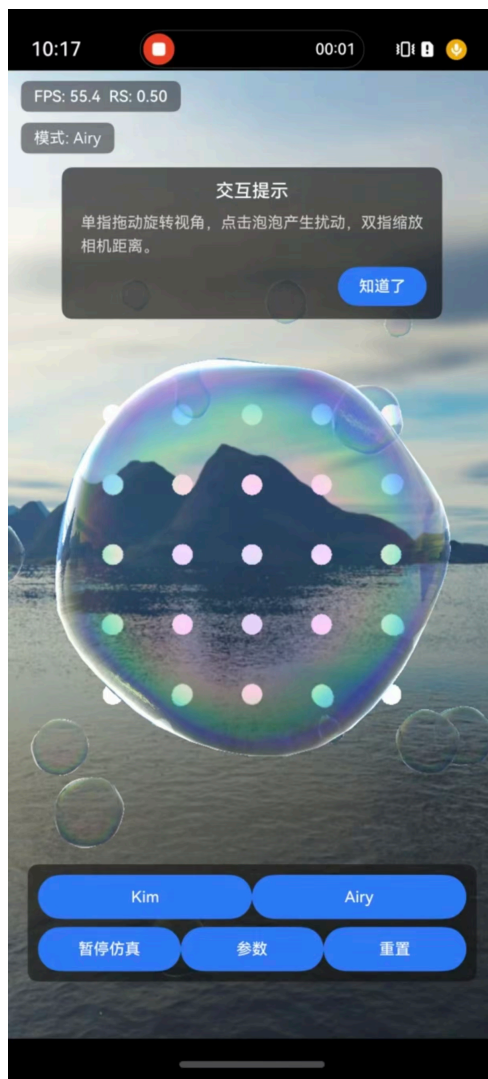


图 1.1 最终整体效果：主泡泡、背景折射、天空盒环境、虹彩边缘以及多个漂移展示泡泡。

1.2 系统组成

最终鸿蒙提交版本的核心源码位于：

HarmonyOS_Source_Submission/refraction/

表 1.1 列出了和实现相关的各个模块。

表 1.1 系统模块与职责

模块	主要内容	作用
页面与交互	ArkTS 页面、XComponent、按钮与参数面板	承载渲染画面，处理触控输入、模式切换、状态显示和参数调节。
Native 接口与管线	NAPI、EGL、FBO、绘制顺序	连接 ArkTS 与 C++ 渲染层，组织多阶段折射渲染和场景合成。
泡泡材质	折射 shader、薄膜虹彩、动态膜厚	实现屏幕空间折射、Kim2012/LUT/Belcour Airy 三种虹彩模式和触摸波纹。
渲染封装	相机、模型、网格、shader 工具类	管理天空盒、球体、动态网格和 OpenGL ES 绘制细节。
表面模拟	DBSTT/vortex sheet 单泡泡动态	让主泡泡轮廓和法线随时间产生轻微形变。
资源文件	天空盒、薄膜 LUT、HAP 提交资源	提供环境反射、背景折射参照、预计算薄膜反射数据和移动端运行素材。

1.3 运行环境

本项目初期开发在 Windows 主机上进行，验证设计可行性和效果后使用 DevEco Studio 完成基于 HarmonyOS 架构的实现。推荐环境如下：

表 1.2 推荐编译与运行环境

项目	要求
开发主机操作系统	Windows 11。
开发工具	DevEco Studio 6.0.2。
SDK 环境	HarmonyOS SDK 6.0.2 (API 22)。
页面层	ArkTS + XComponent。
Native 层	CMake + C++ + EGL + OpenGL ES 3.0 + NAPI。
运行环境	HarmonyOS 真机或本地模拟器。

命令行构建可使用 Hvigor，例如：

```
node "E:\huawei\DevEco_Studio\tools\hvigor\bin\hvigorw.js"
  assembleHap --no-daemon
```

签名完成后，构建产物通常位于：

```
entry/build/default/outputs/default/entry-default-signed.hap
```

提交的文件中已经整理出 HarmonyOS_Source_Submission 下的“可执行程序”目录。

第2章 渲染管线设计

2.1 渲染管线

2.1.1 多阶段屏幕空间折射

实时折射如果完全按路径追踪计算，需要跟踪光线穿过薄膜前表面、后表面以及环境的多次传播，开销较大。项目采用 Wyman 的 image-space refraction 思路^[1]：先把可被折射的背景渲染成纹理，再在泡泡 shader 中根据法线和视线方向扰动采样坐标，从而得到近似折射效果。为了在鸿蒙端同时组织主泡泡、展示泡泡和前后景关系，系统在 Native 层采用多阶段 FBO 合成，但核心光学思路仍然是用两个屏幕空间界面近似肥皂泡的双界面折射。

当前版本的三阶段屏幕空间折射流程

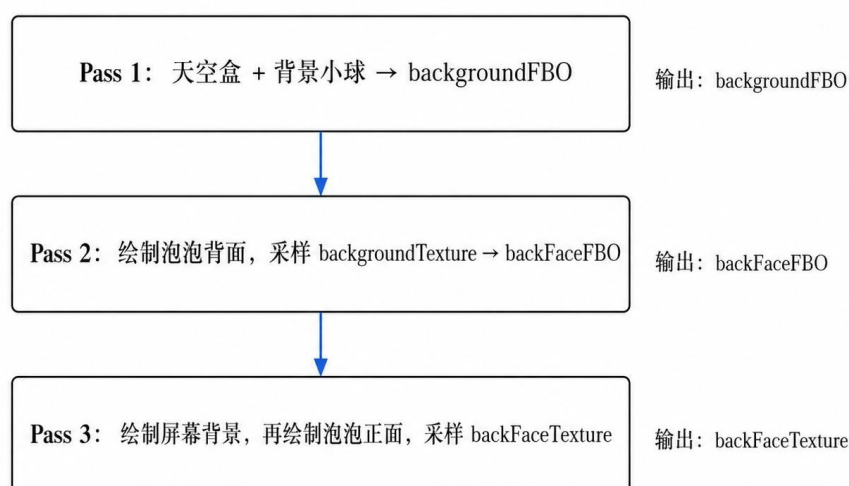


图 2.1 屏幕空间折射主线与多阶段 FBO 合成关系。

这一主线被组织为以下几个连续阶段：

1. **背景阶段**：将天空盒和背景参照物渲染到 backgroundFBO。
2. **后景阶段**：把主泡泡后方场景写入 sceneBehindFBO，为背面折射提供输入。
3. **背面阶段**：开启正面剔除，只绘制主泡泡背面，并输出到 backFaceFBO。
4. **主场景阶段**：绘制包含主泡泡的场景到 mainSceneFBO，供展示泡泡采样。
5. **最终合成阶段**：将主泡泡与多展示泡泡合成到 finalSceneFBO，再输出到屏幕。

这种方法的优点是完全运行在 GPU rasterization 管线中，不需要逐像素射线求交；缺点是折射对象必须已经出现在屏幕空间纹理中，因此它是视觉上可信的近似，而不是严格物理光线追踪。对于鸿蒙终端场景而言，这种取舍能够在移动端预算下同时满足折射观感、多泡泡层次和交互响应速度。

2.1.2 FBO 尺寸策略与坐标修正

系统在 InitFBO 中采用低分辨率 FBO 合成，再由全屏四边形放大到最终屏幕，以兼顾移动端性能与画面完整性。其核心策略可写为：

$$W_{\text{fbo}} = 0.5W_{\text{screen}}, \quad H_{\text{fbo}} = 0.5H_{\text{screen}}.$$

由于 FBO 分辨率与窗口分辨率不同，正面渲染到屏幕时，gl_FragCoord 是窗口像素坐标；背面渲染到 FBO 时，gl_FragCoord 是 FBO 像素坐标。项目通过 uRenderToFBO 区分两种情况：

```
vec2 fboPixel = (uRenderToFBO == 1)
    ? screenPixel
    : screenPixel * (uFBOSize / uWinResolution);
```

该修正对多泡泡展示尤其重要。多展示泡泡在不同 pass 中复用同一套折射与虹彩 shader，如果输入坐标不能正确映射到 FBO 空间，就会出现采样越界或局部发黑。修正后，主泡泡和展示泡泡都沿用统一材质逻辑，并通过模型矩阵、局部几何扰动强度、虹彩模式、输出透明度以及是否响应交互等参数来区分。

2.2 屏幕空间折射实现

2.2.1 折射方向

在 fragment shader 中，入射方向取视线方向 eye，法线取世界法线 normal。肥皂液近似水的折射率，程序使用

$$\eta = \frac{n_{\text{air}}}{n_{\text{film}}} = \frac{1.0}{1.33}.$$

GLSL 中调用 refract(eye, normal, eta) 得到折射方向，并转换到视图空间：

```
vec3 refractVec = mat3(vView) * refract(eye, normal, iorRatio)
;
```

随后取折射方向的 xy 分量作为屏幕空间偏移方向。因为这是屏幕空间近似，shader 并不计算真实光线与后表面的三维交点，而是把折射方向转化为背景纹理采样偏移。

2.2.2 边缘增强与薄膜厚度影响

肥皂泡的中心通常接近透明，轮廓处视觉效果更明显。程序用

$$e = 1 - |\mathbf{V} \cdot \mathbf{N}|$$

表示边缘程度，其中 $\mathbf{V}$ 是视线方向， $\mathbf{N}$ 是表面法线。随后通过 smoothstep 得到更平滑的边缘曲线：

$$e_{\text{profile}} = \text{smoothstep}(0.18, 0.92, e).$$

折射偏移的核心形式可写为：

$$\Delta \mathbf{p} = \mathbf{T}_{xy} s_{\text{ref}} s_{\text{face}} s_{\text{film}} r_{\text{px}} s_{\text{edge}} s_{\text{touch}},$$

其中 $\mathbf{T}_{xy}$ 是视图空间折射方向的屏幕分量， s_{ref} 是用户可调的 uRefractionStrength， s_{face} 区分正面和背面， s_{film} 由薄膜厚度和边缘程度共同决定， r_{px} 是泡泡屏幕半径， s_{edge} 是边缘增强， s_{touch} 是局部触摸扰动增强。

背面 pass 使用更小的折射强度：

$$s_{\text{face}} = \begin{cases} 0.16, & \text{背面 pass,} \\ 1.0, & \text{正面 pass.} \end{cases}$$

这样做可以避免双界面叠加时背景被过度扭曲。最终偏移还会被 uMaxOffsetRatio 限制在泡泡屏幕半径的一定比例内，防止采样跳变。



图 2.2 背景小球经过主泡泡后的折射扭曲效果

2.3 薄膜虹彩模型

2.3.1 薄膜干涉基础

肥皂泡薄膜的厚度通常为数百纳米，和可见光波长处在同一数量级。光线在空气-薄膜界面与薄膜-空气界面发生反射，两个反射波之间产生光程差。对某一波长 λ ，相位差可写为：

$$\phi_\lambda = \frac{4\pi n d \cos \theta_t}{\lambda},$$

其中 n 是薄膜折射率， d 是薄膜厚度， θ_t 是薄膜内部折射角。不同波长的反射强度随 ϕ_λ 改变，因此 RGB 颜色会随厚度、视角和法线方向变化。

项目中所有虹彩模式都使用动态厚度 d 与视角项 $|\mathbf{V} \cdot \mathbf{N}|$ 作为输入，但三种模式计算反射率的方式不同。

2.3.2 Kim2012 模式：实时 RGB 波长采样

程序界面中的 **Kim2012** 模式对应 `uIridescenceMode = 0`。它参考 Kim 与 Park 关于实时肥皂泡虹彩渲染的做法^[2]，在 shader 中直接计算薄膜对 RGB 三个代表波长的反射率。实现文件为 `shader_refraction.h` 中的：

- `fresnelCoefficients`
- `interferencePhase`
- `thinFilmReflectance`
- `kim2012Iridescence`

对每个波长，程序先计算 s 偏振和 p 偏振的 Fresnel 振幅系数，再用相位差得到反射率近似：

$$R_\lambda = \frac{r_s^2(1 - \cos \phi)}{1 + r_s^4 - 2r_s^2 \cos \phi} + \frac{r_p^2(1 - \cos \phi)}{1 + r_p^4 - 2r_p^2 \cos \phi}.$$

当前版本采样的三个代表波长为：

$$\lambda_R = 615\text{nm}, \quad \lambda_G = 535\text{nm}, \quad \lambda_B = 465\text{nm}.$$

该模式的优点是公式直接、易解释，能清楚体现薄膜厚度和相位差对颜色的影响；不足是只用三个波长代表完整可见光谱，边缘颜色可能更鲜艳，也更容易出现强烈的 RGB 分离。

2.3.3 Spectral LUT 模式：预计算薄膜反射纹理

Spectral LUT 模式对应 `uIridescenceMode = 1`。它参考 Iwasaki 等人使用预计算薄膜反射纹理实现实时肥皂泡渲染的思想^[3]。程序不在运行时逐波长计算，而是读取 `uThinFilmLUT`：

```
float thicknessNorm = clamp((thickness - 100.0) / 800.0, 0.0, 1.0);
vec3 encodedReflectance =
    texture(uThinFilmLUT, vec2(clamp(NdotV, 0.0, 0.99),
    thicknessNorm)).rgb;
```

LUT 的横轴为视角项 `NdotV`，纵轴为归一化厚度。该模式运行开销低，颜色更依赖预计算纹理，适合稳定快速的展示；不足是可调范围受 LUT 分辨率和生成方式限制，若想改变折射率、波长积分方式或颜色映射，需要重新生成纹理。

2.3.4 Belcour Airy 模式：多波长 Airy 反射近似

Belcour Airy 模式对应 `uIridescenceMode = 2`，也是当前默认模式。它参考 Belcour 与 Barla 对薄膜虹彩和 Airy reflectance 的建模思想^[4]，但本项目没有完整实现论文中的微表面解析预积分，而是在肥皂泡 `shader` 中实现了一个面向移动端实时展示的简化 Airy 反射模型。

对某一偏振方向，薄膜多次内反射的复振幅可写为：

$$A_{\lambda} = r_{12} + \frac{t_{12}r_{23}t_{21}e^{i\phi_{\lambda}}}{1 - r_{21}r_{23}e^{i\phi_{\lambda}}}, \quad R_{\lambda} = |A_{\lambda}|^2.$$

程序分别计算 `s`、`p` 两个偏振方向，再取平均。随后在 `belcourAiryWeightedIridescence` 中采样多个波长：

430, 470, 500, 535, 575, 615, 650nm.

这些波长再乘以经验 RGB 权重得到最终颜色。相比 Kim2012 模式，Belcour Airy 模式的颜色更平滑，环境反射也会被薄膜反射率染色，因此更适合作为最终鸿蒙版本的默认外观。

2.3.5 三种模式对比

表 2.1 三种虹彩模式的区别

模式	计算方式	优点	局限
Kim2012	shader 中实时计算 RGB 三个代表波长的薄膜干涉反射率。	公式直观，便于说明相位差、Fresnel 系数和厚度的关系。	只采样三个波长，颜色可能偏强，光谱连续性有限。
Spectral LUT	从预计算薄膜反射纹理中按视角和厚度采样 RGB 结果。	运行开销最低，适合实时稳定展示。	外观依赖 LUT，改变物理参数需要重新生成纹理。
Belcour Airy	使用 Airy 风格的多次内反射公式，并采样多个波长加权合成。	颜色过渡更平滑，边缘和环境反射更自然，是当前默认模式。	是项目级简化实现，不是完整 Belcour 微表面预积分模型。

综合比较之下 Airy 模式的实现效果最好，因此最终程序中以 Airy 模式的效果展示为主，另外也同时保留了 Kim 的实现效果作为对比。

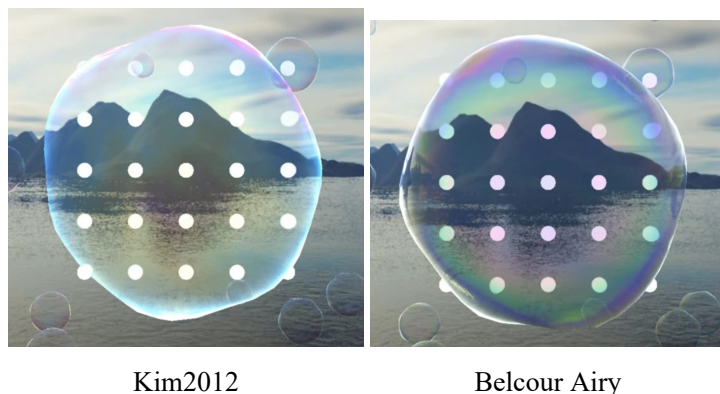


图 2.3 不同虹彩模式对比

2.4 动态膜厚与颜色合成

2.4.1 动态膜厚

真实肥皂泡的膜厚会因为重力排液、表面流动和扰动而变化。完整的膜厚动力学可以基于表面上的流体方程建模^[5]。本项目为了满足移动端实时演示要求，采用 shader 中的程序化近似：使用世界法线方向作为膜面参数，叠加多层 Simplex noise、缓慢流动项、细节噪声和上下方向的排液趋势。

动态厚度可概括为：

$$d = d_0 + A_{\text{var}}P(\mathbf{n}, t) + 190M_{\text{touch}} + 95Q_{\text{ripple}},$$

其中 d_0 是基础膜厚， A_{var} 是膜厚变化幅度， $P(\mathbf{n}, t)$ 是由多层噪声和排液项组合的厚度图案， M_{touch} 是触摸中心的局部增强， Q_{ripple} 是触摸拖动产生的环形波纹。

一个重要实现细节是：项目使用 `normalize(vWorldNormal)` 作为噪声输入方向，而不是使用 UV 坐标。这样可以避免球体 UV 接缝处出现明显断裂，让虹彩流动更连续。

2.4.2 颜色合成

泡泡最终颜色由三部分组成：

1. **透射背景**：从背景 FBO 或背面 FBO 中折射采样得到 `bgColor`。
2. **薄膜虹彩**：由当前模式计算得到 `filmReflectance`，再根据 Fresnel 边缘项增强。
3. **环境反射和高光**：通过 `cubemap` 采样环境反射，并用薄膜反射率染色；另外加入少量 Blinn-Phong 高光。

中心区域透明度更高，轮廓处透明度下降、反射和虹彩增强。shader 中用

$$F = (1 - |\mathbf{V} \cdot \mathbf{N}|)^p$$

作为 Fresnel 边缘项，其中 p 对应可调参数 `uFresnelPower`。这使泡泡中心能保持清透，边缘能呈现更明显的彩色轮廓。

第3章 物理模拟与动力学

3.1 基于涡流片的单泡泡物理模拟

肥皂泡的本质是浸没在环境空气中的一个极薄液膜，其动力学行为主要由液膜的表面张力、周围空气的不可压缩性，以及空气的惯性主导。传统的基于体积的多相纳维-斯托克斯（Navier-Stokes）求解器虽然能够精确模拟这些现象，但需要大量的计算资源来解析泡泡内部的空气动力学以及薄至微米级的薄膜几何形状。

本项目的方方法参考 Da 等人提出的 *Double Bubbles Sans Toil and Trouble: Discrete Circulation-Preserving Vortex Sheets for Soap Films and Foams*^[6] 的思路，仅将建模工作集中在泡泡表面。该论文首次提出了一种基于非流形涡流片（vortex sheet）方程来模拟肥皂膜、泡泡和泡沫动力学的数值方法。本项目在其理论上实现了一个简化版本，专门用于单个闭合泡泡的动态表面演化。

3.1.1 涡流片模型的理论基础

涡流片是一个浸没在三维空间中的表面，它界定了速度场切向分量的不连续性——穿过薄片时，速度的切向分量发生跳跃，而法向分量保持连续。这一数学结构恰好契合肥皂膜的物理本质：薄膜两侧分别是内部空气和外部空气，两侧的速度场可以不同，但空气不能穿透薄膜。

DBSTT 方法使用一个标量场——**环量（circulation）** Γ ——作为描述涡流片状态的主要变量。对于表面上的每一个点 $f(X)$ ，其环量值定义为沿着一个从固定参考点 x_0 到该点、且穿透薄片的任意闭合环路的环路积分：

$$\Gamma(X) = \oint_L \mathbf{u} \cdot d\mathbf{x} \quad (3.1)$$

其中 $\mathbf{u}$ 为环境空气的速度场。这一积分的值与环路 L 的具体形状无关（只要它仅在 x_0 和 $f(X)$ 两处穿透薄片），这保证了 Γ 是表面上的良定义的标量场。

给定环量场之后，可以分两步恢复速度场。首先计算涡流片强度 γ ：

$$\gamma = \mathbf{n} \times \nabla_f \Gamma \quad (3.2)$$

其中 $\mathbf{n}$ 为表面单位法线， ∇_f 为表面梯度算子。

接着，通过 Biot-Savart 积分恢复环境速度场：

$$\mathbf{u}(\mathbf{x}) = \frac{1}{4\pi} \int_S \gamma(\mathbf{x}') \times \frac{\mathbf{x} - \mathbf{x}'}{|\mathbf{x} - \mathbf{x}'|^3} df' \quad (3.3)$$

$\mathbf{u}$ 自动满足 $\nabla \cdot \mathbf{u} = 0$ 。这意味着基于涡流片运动学导出的速度场能够自然地捕捉不可压缩流体的动力学特征，而无需像传统网格方法那样在每步显式求解压力-泊松方程来强制不可压缩性。

下面考虑表面张力驱动的环境演化。在不受外力作用时，根据开尔文环量定理，环量随物质平流守恒： $D\Gamma/Dt = 0$ 。而由于表面张力的存在，环量场会按照一个简洁的局部定律演化。

考虑不可压缩欧拉方程在涡流片两侧的极限形式，结合表面张力产生的压力跳跃 $p_1 - p_2 = \sigma(H_1 - H_2)$ （其中 σ 为表面张力系数， H_1, H_2 分别为薄膜两侧的有符号标量平均曲率），可以得到环量演化的核心方程：

$$\frac{D\Gamma}{Dt} = -\frac{\sigma}{\rho}(H_1 - H_2) \quad (3.4)$$

其中 ρ 为空气密度。对于闭合的单个泡泡，两侧的几何重合但朝向相反，因此 $H_1 = -H_2$ ，从而 $H_1 - H_2 = 2H$ 。这意味着**环量的时间变化率与表面的有符号标量平均曲率成正比**——表面曲率大的区域环量变化快，曲率小的区域变化慢。表面张力正是通过这一机制驱动泡泡表面趋向平衡态（对闭合泡泡而言是球面，其上各点曲率相等）。

3.1.2 离散化与数值实现

将上述连续理论转化为数值算法，需要在空间和时间维度上进行离散化。以下详细介绍本项目在 `VortexSheetSimulation` 类中的实现。

使用三角网格来表示泡泡表面，在顶点上采样环量场。项目定义了 `SimMesh` 结构来承载模拟所需的所有数据：

- `positions`: 顶点三维位置 $\mathbf{p}_i \in \mathbb{R}^3$
- `triangles`: 三角形索引三元组 (i_0, i_1, i_2)
- `circulation`: 逐顶点存储的标量环量 $\Gamma_i \in \mathbb{R}$
- `vertexNormals`: 顶点法线 $\mathbf{n}_i$ （通过角度加权平均求出）
- `voronoiAreas`: 顶点 Voronoi 对偶面积 A_i
- `meanCurvature`: 逐顶点有符号标量平均曲率 H_i
- `velocity`: 逐顶点的诱导速度 $\mathbf{u}_i$

初始几何由 `initIcosphere()` 构建：程序首先生成一个单位二十面体，再通过 `midpoint` 方式对 `icosphere` 做指定次数的细分，最后归一化到指定半径 $R = 1.5$ 。为了触发动力学演化，程序沿一个固定拉伸轴施加初始顶点扰动；在当前鸿蒙提交版本中，应用层设置的扰动幅度为 16%。环量场初始化为全零，法线和面积在初始化完成后立即计算。

算法总览

每次仿真更新遵循 DBSTT 论文^[6] Algorithm 1 的时间积分循环，经过本项目简化适配后的子步更新流程为：

1. 几何量计算: `computeVertexNormalsAndAreas()` + `computeMeanCurvature()`
2. 表面张力积分: `integrateSurfaceTension(dt)`
3. 环量扩散: `diffuseCirculation()`
4. 涡流片强度计算: `computeVortexSheetStrength(gamma)`
5. 毕奥-萨伐尔速度求解: `computeBiotSavartVelocities(gamma)`
6. 位置推进: `advectPositions(dt)`

`VortexSheetSimulation` 类内部保留了 `substepsPerFrame = 5`、`timeStep = 0.0005f` 的默认配置，但当前鸿蒙提交版本在初始化与重置时都会显式改写为 `substepsPerFrame = 1`、`timeStep = 0.001f`。同时，Native 渲染层并不是每一帧都推进仿真，而是每隔 `kSimFrameInterval = 6` 帧调用一次 `g_Sim.update(1.0f / 60.0f)`。因此，当前系统中一次实际仿真更新只执行 1 个子步，子步时间步长为 $\Delta t = 0.001$ 秒，子步结束后重新计算法线以供渲染使用。

顶点法线与 Voronoi 面积计算

顶点法线采用角度加权平均法计算。对每个入射到顶点 v 的三角形面，计算该面的单位法线 $\mathbf{n}_f$ ，并将其按该三角形在顶点处的内角加权累加：

$$\mathbf{n}_v = \frac{\sum_{f \in \text{faces}(v)} \alpha_f \cdot \mathbf{n}_f}{\left\| \sum_{f \in \text{faces}(v)} \alpha_f \cdot \mathbf{n}_f \right\|} \quad (3.5)$$

其中 α_f 为面 f 中位于顶点 v 处的内角，通过相邻边向量的点积与反余弦求得。对于退化情况（法线长度接近于零），则退化为从原点指向顶点的径向方向。

Voronoi 面积通过将每个入射三角形的面积三等分并累加到其三个顶点上来近似计算。

离散平均曲率

对于标量平均曲率的离散化，本项目参考论文的建议，采用基于边的离散曲率公式：对每对相邻顶点 (v_i, v_j) （即每条共享两个三角形的边），计算该边上的积分平均曲率，再均匀分配到其两个端点。具体来说，对于连接 v_i, v_j 的边，假设它由两个三角形 t_0, t_1 共享：

$$\mathbf{K}_i = -\frac{1}{4A_i} \sum_{j \in \mathcal{N}(i)} (\cot \alpha_{ij} + \cot \beta_{ij}) (\mathbf{p}_j - \mathbf{p}_i) \quad (3.6)$$

其中 A_i 为顶点 v_i 的 Voronoi 面积， $\mathcal{N}(i)$ 为 v_i 的一环邻域， α_{ij}, β_{ij} 分别为边 e_{ij} 在两个入射三角形中对面的两个角。 $\cot \alpha$ 通过边向量的点积除以叉积模长求

得：

$$\cot \alpha = \frac{\mathbf{e}_{ik} \cdot \mathbf{e}_{jk}}{\|\mathbf{e}_{ik} \times \mathbf{e}_{jk}\|} \quad (3.7)$$

在代码实现中，程序首先构建从边到其入射面列表的映射，然后仅处理恰好由两个三角形共享的边，计算两侧的余切权重并累加到曲率法线 $\mathbf{K}_i$ 上。最终的标量平均曲率通过对曲率法线取负点积获得：

$$H_i = -\mathbf{K}_i \cdot \mathbf{n}_i \quad (3.8)$$

负号确保了向外法线约定下的符号正确性：在凸区域（如球面）， $\mathbf{K}$ 指向内侧，与向外法线 $\mathbf{n}$ 的点积为负，取负后 $H > 0$ 。

表面张力积分

有了每个顶点的离散标量平均曲率 H_i ，就可以根据方程 (3.4) 对环量进行前向欧拉更新。对于单个闭合泡泡，两侧界面几何重合但朝向相反： $H_1 = H, H_2 = -H$ ，故 $H_1 - H_2 = 2H$ 。离散更新公式为：

$$\Delta \Gamma_i = \text{sign} \cdot \frac{\sigma}{\rho} \cdot \Delta t \cdot 2 \cdot H_i \quad (3.9)$$

其中 σ/ρ 对应代码中的 `surfaceTensionStrength`（默认值 15.0），`sign` 通过 `flipSurfaceTensionSign`（默认 `true`，即取正号）控制。

值得注意的是，论文中原本的公式包含顶点面积 A_v 在分母中，但由于我们使用逐点的 H_i 而非积分的曲率 H_i^v ， A_v 在分子分母中恰好抵消简化。

Vortex Sheet Strength 计算

环量更新完成后，需要在每个三角形上计算涡流片强度 γ 。根据方程 (3.2)， $\gamma = \mathbf{n} \times \nabla_f \Gamma$ 。在三角形 $t = (v_0, v_1, v_2)$ 上，环量场是分段线性的，其表面梯度为常数。代码实现使用三角形三个顶点的环量值来计算梯度：

$$\gamma_t = \mathbf{n}_t \times \frac{\Gamma_0(\mathbf{p}_2 - \mathbf{p}_1) + \Gamma_1(\mathbf{p}_0 - \mathbf{p}_2) + \Gamma_2(\mathbf{p}_1 - \mathbf{p}_0)}{2 \cdot \text{area}(t)} \quad (3.10)$$

其中 $\mathbf{n}_t$ 为三角形面的单位法线，通过边向量的叉积归一化得到，且保证与从原点指向三角形质心的方向一致（向外朝向）。

正则化毕奥-萨伐尔速度求解

对于每个顶点 v_i ，需要计算所有三角形 t_j 对其诱导速度的贡献之和。为避免距离趋近于零时的奇异性，DBSTT 采用正则化形式：

$$\mathbf{u}(\mathbf{x}) = \frac{1}{4\pi} \sum_t \gamma_t \times \frac{\mathbf{x} - \mathbf{c}_t}{(|\mathbf{x} - \mathbf{c}_t|^2 + \alpha^2)^{3/2}} \cdot A_t \quad (3.11)$$

其中 $\mathbf{c}_t$ 为三角形 t 的质心， A_t 为其面积， α 为正则化参数。代码中 α 自动设为网格平均边长（在初始化时计算）的 1.0 倍，即 `regularizationAlpha = avgEdgeLength * 1.0f`。

实现细节上，程序预计算每个三角形的质心、面积和 γ_t 值，然后对每个顶点遍历所有三角形，进行双重循环求和。每个贡献项为 $\gamma_t \times \mathbf{r}$ 乘以面积和分母幂次的倒数，其中 $\mathbf{r} = \mathbf{x} - \mathbf{c}_t$ 为从三角形质心到当前顶点的向量。最终速度除以 4π 以匹配连续积分中的系数。

位置推进与安全钳制

顶点位置使用前向欧拉法更新： $\mathbf{p}_i \leftarrow \mathbf{p}_i + \mathbf{u}_i \cdot \Delta t$ 。为确保稳定性，代码施加了一个位移上限，限制每个子步的顶点位移不超过平均边长的 10%：

$$\|\Delta \mathbf{p}_i\|_{\max} = 0.1 \cdot \bar{e} \quad (3.12)$$

其中 $\bar{e}$ 为平均边长。这是为了解决调试代码过程中出现的因数值不稳定导致的网格穿刺或顶点飞散问题。

位置更新后，所有顶点统一减去网格质心，以防止因数值漂移导致的整体平移，保持毕奥-萨伐尔积分的对称性。

环量扩散

为了进一步抑制数值不稳定性，代码在每个子步中施加一个轻微的拉普拉斯扩散：

$$\Gamma_i \leftarrow \Gamma_i + \nu \cdot \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} (\Gamma_j - \Gamma_i) \quad (3.13)$$

其中 ν 为扩散系数，默认值 `circulationDiffusion = 0.0005`，非常小，仅用于消除高频数值振荡而不影响宏观动力学行为。DBSTT 论文也指出，这种扩散可以解释为对空气粘度的合理视觉近似。

3.1.3 与本项目渲染管线的集成

物理模拟与渲染管线的衔接位于 `RenderFrame()` 函数中：

1. 渲染循环开始后，若模拟未被暂停（`g_SimPaused == false`），则每隔 `kSimFrameInterval = 6` 帧调用一次 `g_Sim.update(1.0f/60.0f)`，按较低频率推进一次仿真更新。
2. 模拟完成后，调用 `buildVerticesFromSim()` 将 `SimMesh` 中的顶点位置和法线转换为 GPU 可用的 `Vertex` 数组。
3. 通过 `Mesh::updateVertices()` 将更新后的顶点数据上传至 OpenGL VBO，后续的折射/虹彩 `shader` 即可使用变形后的几何体进行渲染。

主泡泡的模型矩阵保持为单位矩阵，所有变形信息完全包含在经模拟更新的顶点位置中，这使得 `shader` 中的法线计算能够正确地反映泡泡表面的动态形变。

当前实现没有加入原论文中的完整非流形拓扑变化、泡沫多区域标记和线框约束，只是一个 DBSTT 风格的单泡泡表面动态近似。它的作用是让主泡泡轮廓和表面法线随时间发生微小变化，从而让折射和虹彩不显得完全静止。

具体动态效果详见项目演示视频。可以明显观察到主泡泡在当前系统中保持持续的小幅表面形变，并在参数调整后呈现不同强度的恢复趋势。

3.2 触摸交互与局部光学响应

除了自动的表面动力学演化之外，本项目还实现了模拟真实触碰泡泡的交互效果。交互设计并不把触摸输入简单地作为全局参数滑杆，而是将其解释为用户在屏幕空间中接触泡泡表面所产生的局部扰动。系统中，触摸事件通过 ArkTS 的 `onTouch` 回调转发到 Native 层：单指拖动主要用于旋转观察视角，单指点击、短按以及未触发视角旋转阈值的微小移动用于触发主泡泡的局部触摸响应，双指缩放用于调整相机距离；在 `shader` 层，这些输入仍统一被表述为触摸点、触摸强度和触摸速度。

具体来说，程序将触摸位置归一化为屏幕空间坐标 `uTouchPoint`，并根据按压状态和局部移动速度构造两个交互参数：触摸强度 `uTouchStrength` 与触摸速度 `uTouchVelocity`。在 `fragment shader` 中，对当前片元屏幕坐标 $\mathbf{x}$ 与触摸点 $\mathbf{x}_{touch}$ 的距离建立一个局部高斯影响区域：

$$M_{touch} = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_{touch}\|^2}{r^2}\right) \cdot S_{touch}, \quad (3.14)$$

其中 S_{touch} 对应触摸强度， r 控制局部作用半径。为了表现手指接触薄膜后产生的环形扰动，`shader` 还叠加了一个随距离衰减的波纹项：

$$Q_{ripple} = \sin(k\|\mathbf{x} - \mathbf{x}_{touch}\| - \omega t) \exp(-\lambda\|\mathbf{x} - \mathbf{x}_{touch}\|) \cdot S_{touch}. \quad (3.15)$$

触摸项会同时影响折射和色散两个视觉通道。首先，在屏幕空间折射部分，局部触摸会放大原有的折射偏移，使泡泡背后的背景在触摸点附近发生更明显的扭曲：

$$\Delta \mathbf{p}_{touch} = \Delta \mathbf{p}_{base} \cdot (1 + 2.4M_{touch} + 1.1|Q_{ripple}|) + \hat{\mathbf{d}}_{touch} Q_{ripple}. \quad (3.16)$$

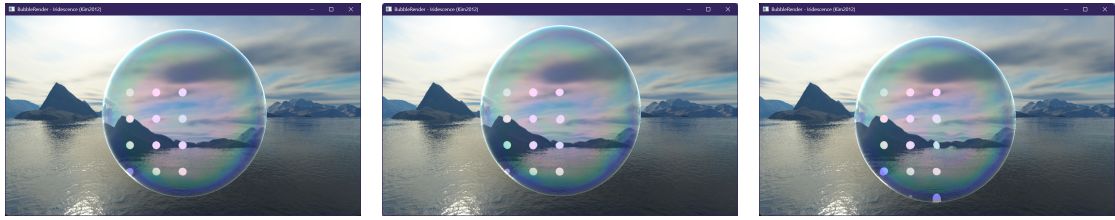
其中 $\Delta \mathbf{p}_{base}$ 是基础折射偏移, $\hat{\mathbf{d}}_{touch}$ 是从触摸点向当前片元的屏幕空间方向。这样, 触摸不会改变整个泡泡的折射强度, 而是在接触区域形成局部的折射增强, 更接近真实泡泡受到扰动时的视觉表现。

其次, 触摸扰动也会改变薄膜厚度场。由于薄膜干涉颜色对膜厚变化非常敏感, 局部膜厚扰动会直接表现为虹彩颜色和色散强度的变化。本项目在原有噪声膜厚场上加入触摸项:

$$d = d_0 + A_{var}P(\mathbf{n}, t) + 190M_{touch} + 95Q_{ripple}, \quad (3.17)$$

其中 $P(\mathbf{n}, t)$ 是随法线方向和时间变化的程序化膜厚扰动。触摸点附近的 M_{touch} 使局部膜厚发生偏移, Q_{ripple} 则形成环状传播的薄膜厚度变化, 从而让触摸区域出现更强的局部虹彩和色散。

图 3.1 展示了触摸交互的三个典型状态。无触摸时, 泡泡只显示基础折射与薄膜虹彩; 单指点击或按压后, 接触点附近的背景扭曲和颜色变化明显增强; 在不触发视角旋转的轻微移动情况下, 触摸速度会进一步增强波纹项, 使局部折射与色散出现更明显的动态响应。



无触摸

按下触摸

轻微位移波纹

图 3.1 触摸交互对泡泡局部折射和色散的影响。系统中的触摸输入不会作为全局参数调节器, 而是在触摸点附近产生局部薄膜扰动, 从而增强背景折射变形与虹彩色散。

3.3 多泡泡展示与风吹漂移

为了增强展示效果, 项目加入了 16 个展示泡泡。它们不是独立的物理模拟泡泡, 而是使用静态球体模型、同一套折射/虹彩 shader, 以及独立的模型矩阵动画。每个展示泡泡包含:

- 基础位置 `basePosition`
- 半径 `radius`
- 相位 `phase`
- 风向振幅 `windAmplitude`
- 上下浮动振幅 `floatAmplitude`
- 速度 `speed`

风吹动画由 `BubbleModelMatrix` 计算, 简化表达为:

$$\mathbf{x}_i(t) = \mathbf{x}_{i,0} + \mathbf{w} \sin(s_i t + \phi_i) A_i + \mathbf{y} \sin(0.73 s_i t + 1.7 \phi_i) B_i + \mathbf{z} \cos(0.51 s_i t + 0.9 \phi_i) C_i.$$

其中 $\mathbf{w}$ 是归一化风向， $\mathbf{y}$ 是竖直方向， $\mathbf{z}$ 是深度方向。半径还会乘以轻微 wobble 系数，让泡泡不是完全刚性缩放。

展示泡泡的排布包含前景和背景：部分泡泡的 z 坐标在主泡泡前方，用来增加层次感；另一部分分布在左右和后方，用来形成更丰富的空间构图。它们与主泡泡使用相同 shader，因此当前版本的副泡泡不会再单独走一套容易产生黑色问题的材质逻辑。实现时，程序会先按展示泡泡到相机的距离从远到近排序，再以固定输出透明度进行 Alpha 混合合成，从而减轻透明物体互相覆盖时的排序伪影。

图 3.2 展示了两个不同时刻的多泡泡位置变化；更连续的浮动和轻微形变效果可参考项目 demo 视频。



时刻一

时刻二

图 3.2 16 个展示泡泡在风吹动下轻微漂移，部分位于主泡泡前景，部分位于背景。

第4章 交互与演示

4.1 交互实现

4.1.1 触控交互

当前系统以 HarmonyOS 触控为主要交互方式，最核心的演示路径由三类手势构成：

表 4.1 触控交互

操作	效果
单指拖动	改变相机 yaw/pitch，围绕泡泡观察。
双指缩放	改变相机距离，实现缩放。
单指点击/短按	在主泡泡上产生局部膜厚增强和环形折射波纹。

在页面层，ArkTS 会先根据 XComponent 的实际区域将输入事件映射到局部坐标，再通过 NAPI 把坐标、事件类型和视口尺寸传给 Native 渲染层。单指点击主泡泡时，程序把触摸位置转换为归一化屏幕坐标 $g_TouchPoint$ ，并记录触摸强度与局部移动速度 $g_TouchVelocity$ 。当单指移动超过较小阈值后，ArkTS 侧会优先把后续移动解释为 `rotateCamera()` 输入，因此连续大幅拖动并不是局部波纹的主要触发方式。shader 中计算当前 fragment 到触摸点的距离：

$$r = \|\mathbf{u}_{frag} - \mathbf{u}_{touch}\|.$$

局部触摸强度采用高斯型衰减：

$$M_{touch} = \exp\left(-\frac{r^2}{0.0065}\right) S_{touch},$$

环形波纹采用正弦函数叠加指数衰减：

$$Q_{ripple} = \sin(78r - 18t) \exp(-9r) S_{touch}(0.35 + 0.65V_{touch}).$$

触摸只作用在主泡泡上。绘制展示泡泡时，程序向 shader 传入无效触摸点和零强度，避免副泡泡受到局部触摸扰动。由于这一部分的效果高度依赖连续交互，具体动态表现以比赛提交视频中的演示为准。

4.1.2 界面参数

系统将主要控制项整理为移动端界面按钮和参数面板，使评审能够直接在鸿蒙设备上观察到参数变化。当前界面主要包含模式切换、仿真控制和参数滑条

三类入口：

表 4.2 界面参数控制

界面项	参数	作用
模式按钮	Kim / Airy	切换两种主要演示模式；Native 层仍保留 Spectral LUT 便于内部对比。
状态按钮	暂停仿真 / 重置	暂停或继续主泡泡表面模拟，并同步冻结或恢复基于时间的视觉变化；重置时恢复默认参数。
光学参数	干涉膜厚、折射强度、环境反射	调整彩色条纹尺度、背景扭曲强度和環境反射权重。
仿真参数	厚度扰动、表面张力、边缘畸变	调整动态膜厚幅度、主泡泡恢复趋势和轮廓额外折射增强。
视图参数	相机距离	调整观察远近，便于展示单泡泡细节或多泡泡整体构图。
状态显示	FPS、当前模式、交互提示	方便现场展示系统帧率、当前材质模式和操作说明。

除 ArkTS 界面当前开放的滑条外，Native 接口层还保留了 uFresnelPower 与 uMaxOffsetRatio 对应的调节入口，用于内部调参与折射稳定性验证；它们没有直接暴露在比赛演示面板中。

这些参数直接服务于最终鸿蒙版本的展示与调参流程。具体调节过程和实时反馈可参考演示视频。

第 5 章 结果与分析

5.1 结果分析与局限

当前系统已经实现了面向比赛提交和现场演示的 HarmonyOS 透明泡泡渲染效果。主要结果包括：

- 通过多阶段 FBO 近似双界面折射，背景参照物和天空盒能在泡泡中产生明显扭曲。
- 三种薄膜虹彩模式已经在 Native 层实现，比赛界面重点开放 Kim 与 Airy 两种展示模式，便于说明实时公式与多波长近似之间的差异。
- 动态膜厚让虹彩随时间缓慢流动，不依赖静态纹理。
- 触摸交互能够产生局部膜厚增强和波纹，配合参数面板形成更自然的现场展示反馈。
- 主泡泡接入 DBSTT/vortex sheet 风格表面动态，16 个多展示泡泡通过距离排序与 Alpha 混合合成，用于丰富空间构图与比赛演示层次。
- ArkTS 页面、NAPI 接口、Native 渲染层和可执行 HAP 程序已经形成完整的提交链路。

同时，系统也有明确局限：

- 屏幕空间折射不能表现屏幕外物体，也不能严格模拟多泡泡之间的相互折射。
- Belcour Airy 模式是实时 shader 中的项目级简化，不包含完整微表面模型和解析光谱预积分。
- 动态膜厚主要由噪声、排液趋势和触摸扰动驱动，没有求解完整的薄膜厚度流体方程。
- DBSTT 模拟只用于单个闭合泡泡，不支持原论文中的泡沫拓扑变化和 Plateau 边界。
- 展示泡泡只做视觉漂移，不参与主泡泡的物理碰撞或流体耦合。

如果继续迭代比赛版本，后续优先考虑补强多泡泡之间的相互折射、碰撞与排序一致性，并进一步完善移动端性能记录。

5.2 总结

本项目从透明肥皂泡的主要视觉现象出发，实现了一个面向赛题二提交的 HarmonyOS 渲染系统。渲染方面，项目通过多阶段 FBO 合成近似双界面折射，并用低分辨率 FBO 与坐标修正兼顾移动端性能和多泡泡显示稳定性。光学方面，

项目实现了 Kim2012、Spectral LUT、Belcour Airy 三种薄膜虹彩模式，能够展示实时公式计算、预计算纹理和多波长 Airy 近似之间的差异。动态方面，项目结合程序化膜厚变化、触摸波纹、DBSTT 风格主泡泡形变和多泡泡风吹漂移，使最终结果更符合比赛对“效果、交互、完整性”的要求。

后续如继续扩展该项目，可进一步加入更严格的光谱到 RGB 积分、更真实的膜厚流体模拟、多泡泡之间的相互折射和碰撞，以及基于体积或拓扑变化的泡沫模拟。

参 考 文 献

- [1] WYMAN C. An approximate image-space approach for interactive refraction[C]//ACM SIG-GRAPH 2005 Sketches. ACM, 2005: 1050-1053.
- [2] KIM N, PARK K. Real time rendering iridescent colors appearing on soap bubbles[M/OL]//Lecture Notes in Electrical Engineering: volume 181 Embedded and Multimedia Computing Technology and Service. Springer, 2012: 377-384. DOI: 10.1007/978-94-007-5075-3_45.
- [3] IWASAKI K, MATSUZAWA K, NISHITA T. Real-time rendering of soap bubbles taking into account light interference[C/OL]//Proceedings of Computer Graphics International. IEEE, 2004: 344-348. DOI: 10.1109/CGI.2004.1309237.
- [4] BELCOUR L, BARLA P. A practical extension to microfacet theory for the modeling of varying iridescence[J/OL]. ACM Transactions on Graphics, 2017, 36(4): 65:1-65:14. DOI: 10.1145/3072959.3073620.
- [5] ISHIDA S, SYNAK P, NARITA F, et al. A model for soap film dynamics with evolving thickness[J/OL]. ACM Transactions on Graphics, 2020, 39(4): 31:1-31:11. DOI: 10.1145/3386569.3392405.
- [6] DA F, BATTY C, WOJTAN C, et al. Double bubbles sans toil and trouble: Discrete circulation-preserving vortex sheets for soap films and foams[J/OL]. ACM Transactions on Graphics, 2015, 34(4): 149:1-149:9. DOI: 10.1145/2767003.